

Dust Powerloss Detection: Development Blog (Weeks 6–9)

This document summarises the four-week development journey across the repository. It maps the project from dataset preparation and feature engineering to augmentation, CNN and transfer learning, and final Flask deployment.

Week 6 focused on dataset understanding, image preprocessing, and feature engineering. I cleaned the dataset, removed white-background images, and created the train/test structure required by the notebooks.

Week 7 centred on augmentation and CNN feature extraction. I used ImageDataGenerator to expand the dataset and then trained a convolutional pipeline that generated meaningful feature maps for classification.

Week 8 advanced the work into transfer learning with MobileNet, Inception, and VGG16. This step allowed me to compare different feature extractors and determine which representation was most suitable for dust detection.

Week 9 focused on evaluation and deployment. I integrated the trained workflow into the Flask app and connected the repository into a usable prediction system.

Week 6 – Data foundation and feature engineering

I started by revisiting the project goal and working through the README, the Solar Panel Dust Detection Notebook, and the Feature Engineering notebook. I used the dataset structure in Detect_solar_dust to label clean and dusty images and then cleaned the data by removing white-background images, cropping the panel area, and organising the dataset into train and test folders. This stage made the learning task more reliable.

Week 7 – Augmentation and CNN training

I moved into augmentation and CNN feature extraction. The Image Augmentation notebook used rotation, flipping, and brightness adjustment to increase robustness. In the CNN + ML Models notebook, I defined a convolutional stack, extracted features, and passed these into classifiers such as Logistic Regression and Random Forest for comparison.

Week 8 – Transfer learning and model comparison

This stage was the deepest technical phase of the work. I used MobileNet, Inception, and VGG16 without the top classification layer, then flattened the learned features and fed them into classical machine learning classifiers. This let me evaluate which feature extractor provided the strongest representation for classifying dusty and clean panels.

Week 9 – Deployment and final integration

In the final stage I moved from experimentation to deployment. The Flask app in cnn_flask_api.py loads a trained model, accepts uploaded images, preprocesses them to the expected size and normalisation, and predicts whether the panel is dusty or clean. This carried the project from notebook research into a usable prototype system.

Snippet 1

Snippet 1 – Feature Engineering / White Background Filtering

```
def remove_white_background_images(path, threshold = 0.2):
    image_paths = []
    for image in tqdm(os.listdir(path)):
        image_path = os.path.join(path, image)
        image_array = cv2.imread(image_path)
        gray = cv2.cvtColor(image_array, cv2.COLOR_BGR2GRAY)
        _, binary_image = cv2.threshold(gray, 240, 255, cv2.THRESH_BINARY)
        white_pixel_count = cv2.countNonZero(binary_image)
        total_pixel_count = binary_image.shape[0] * binary_image.shape[1]
        percentage_of_white_pixels = white_pixel_count / total_pixel_count
        if percentage_of_white_pixels > threshold:
            image_paths.append(image_path)
    return image_paths
```

Snippet 2

Snippet 2 – Image Augmentation

```
data_generator = ImageDataGenerator(  
    rotation_range = 45,  
    horizontal_flip = True,  
    vertical_flip = True,  
    brightness_range = [0.5, 1.5]  
)  
  
# These transformations help the model learn robust patterns  
# despite angle, lighting, and panel orientation variance.
```

Snippet 3

Snippet 3 – CNN Feature Extractor

```
model = Sequential()  
model.add(Conv2D(64, (2, 2), activation='relu', input_shape=INPUT_SHAPE))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Conv2D(32, (2, 2), activation='relu'))  
model.add(MaxPooling2D(pool_size=(2, 2)))  
model.add(Conv2D(16, (2, 2), activation='relu'))  
model.add(Flatten())  
features_train = model.predict(X_train)  
features_cv = model.predict(X_cv)  
features_test = model.predict(X_test)
```

Snippet 4

Snippet 4 – MobileNet / VGG16 Transfer Learning

```
base_model = MobileNet(include_top=False, weights="imagenet", input_shape=(224, 224, 3))
base_model.trainable = False
features_train = base_model.predict(X_train).reshape((len(X_train), -1))
features_cv = base_model.predict(X_cv).reshape((len(X_cv), -1))
features_test = base_model.predict(X_test).reshape((len(X_test), -1))

# The base network is frozen to keep general visual features
# while the classifier learns the dust/no-dust boundary.
```

Snippet 5

Snippet 5 – Flask Prediction Endpoint

```
model = load_model("Models/Mobilenet.h5")
@app.route("/predict", methods=["POST"])
def predict():
    file = flask.request.files["file"]
    file_path = os.path.join(app.root_path, "temp", file.filename)
    file.save(file_path)
    img = cv2.imread(file_path, cv2.IMREAD_COLOR)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = cv2.resize(img, (224, 224))
    predictions = model.predict(x)
    return flask.render_template("result.html", label=label)
```